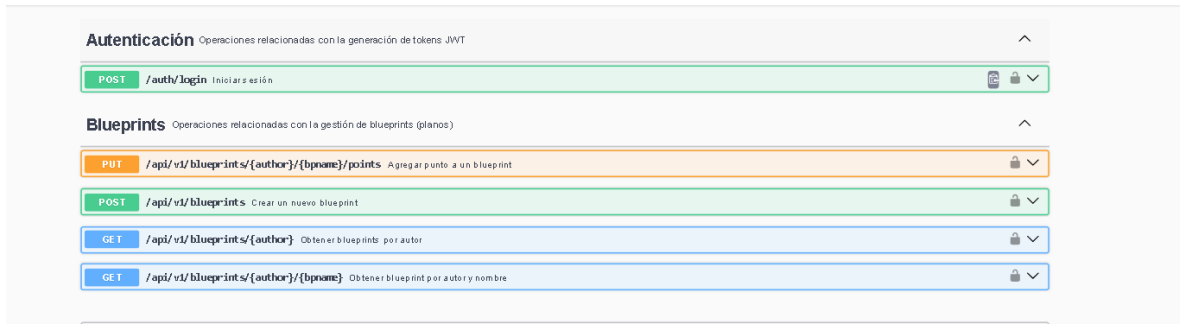


Documentar en Swagger los endpoints de autenticación y de negocio.

Durante el desarrollo del laboratorio se integró y configuró **Swagger (OpenAPI 3)** para documentar de manera clara y estructurada los endpoints de la API REST.



Modificamos las clases con un tag adecuado

```
import java.time.Instant;
import java.util.Map;

@Tag(name = "Autenticación", description = "Operaciones relacionadas con la generación de tokens JWT")
@RestController
@RequestMapping("/auth")
public class AuthController {

    private final JwtEncoder encoder;
    private final InMemoryUserService userService;
    private final RsaKeyProperties props;

    public AuthController(JwtEncoder encoder, InMemoryUserService userService, RsaKeyProperties props) {
        this.encoder = encoder;
        this.userService = userService;
        this.props = props;
    }

    public record LoginRequest(String username, String password) {}

    public record TokenResponse(String access_token, String token_type, long expires_in) {}

    @Operation(summary = "Iniciar sesión", description = "Valida las credenciales del usuario y genera un token JWT firmado")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Autenticación exitosa, token generado"),
        @ApiResponse(responseCode = "401", description = "Credenciales inválidas")
    })
    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest req) {
        if (!userService.isValid(req.username(), req.password())) {
            return ResponseEntity.status(status: 401).body(Map.of(k1: "error", v1: "invalid_credentials"));
        }

        Instant now = Instant.now();
        long ttl = props.tokenTtlSeconds() != null ? props.tokenTtlSeconds() : 3600;
    }
}
```